

Пояснительная записка к решению

**Сервис формирования цифрового признака
транспортного средства для сопоставления
снимков одного автомобиля из разных локаций
без использования государственного номера**

Повторная идентификация ТС по внешнему виду: модель 1.2.0-final (DINOv3 ViT-L/16 + BN neck), офлайн-образ жюри и сервис оператора

Команда	znatalk.ai
Состав	Дворядкин Кирилл — капитан, тимлид Глухов Сергей Владимирович — ML-инженер Вологин Иван Александрович — бэкенд-разработчик
Демо	https://lct.znatalk.ai
Дата	29.09.2026

Содержание

1 Назначение решения и постановка задачи

- 1.1 Задача
- 1.2 Данные конкурса

2 Функциональная архитектура

- 2.1 Пользовательские сценарии
- 2.2 REST API сервиса
- 2.3 Два пути исполнения

3 Компонентная архитектура

- 3.1 Сервисы и контейнеры
- 3.2 Поток запроса «поиск»
- 3.3 Контракты и хранение
- 3.4 Офлайн-образ жюри
- 3.5 Структура репозитория

4 Методы обработки данных и алгоритмы

- 4.1 Предобработка
- 4.2 Энкодер
- 4.3 Обучение: цепочка и гиперпараметры
- 4.4 Роли данных и проверка чистоты
- 4.5 Ранжирование: DBA только по галерее
- 4.6 Режим отказа
- 4.7 Объяснимость

5 Сборка, установка и запуск

- 5.1 Требования
- 5.2 Сервис оператора одной командой
- 5.3 Офлайн-образ жюри
- 5.4 Воспроизведение обучения и калибровки
- 5.5 Тесты и линтеры

6 Метрики на валидации

- 6.1 Протокол
- 6.2 Результаты выбранной модели
- 6.3 История выбора модели

7 Обоснование порога отказа

- 7.1 Постановка
- 7.2 Почему именно так выбирается порог
- 7.3 Результат

8 Скорость

- 8.1 Протокол
- 8.2 Результаты (RTX 3090, драйвер 616.92, Docker Linux, сеть отключена, RTX JIT отключён)

9 Анализ ошибок и ограничения

- 9.1 Характерные ошибки
- 9.2 Пути развития
- 9.3 Ограничения, которые мы не скрываем

10 Масштабирование

- 10.1 ANN на 10^8 векторов
- 10.2 Эталонная архитектура для города

11 Внешние библиотеки, фреймворки и датасеты

12 Команда и ссылки

1 Назначение решения и постановка задачи

1.1 Задача

По ТЗ (тз.txt, разделы 3, 8, 9) сервис должен: (1) формировать эмбединг транспортного средства по кадру и заданному ограничивающему прямоугольнику (bbox); (2) ранжировать галерею по близости к запросу; (3) в режиме предложения кандидатов либо возвращать принятого кандидата с оценкой уверенности, либо отказывать, если уверенного совпадения нет. Детекция ТС, распознавание номеров и трекинг в задачу не входят. Номер, OCR, идентификатор камеры, время и прочие метаданные **не подаются в энкодер**: его входы — только пиксели кадра и bbox.

Решение команды znatalk.ai состоит из двух путей исполнения с одной и той же замороженной моделью **1.2.0-final** (DINOv3 ViT-L/16 + BN neck, 1 024-мерный L2-нормированный вектор):

- **офлайн-образ жюри reid-submit** — одной командой `docker run` превращает каталог кадров и два CSV в три файла сдачи (`submission.csv`, `candidates.csv`, `embeddings.npy`); работает без сети, только на GPU;
- **сервис оператора** — веб-интерфейс, REST API, микросервисы на Rust и PostgreSQL/pgvector; поднимается одной командой `docker compose up --build`; развёрнут на <https://lct.znatalk.ai>.

Все числа в записке взяты из файлов репозитория; рядом с таблицами указан файл-источник. Десятичный разделитель — точка, как в исходных файлах доказательств. Метрики получены на размеченных конкурсных данных, отделённых от обучения, но **не являются результатом скрытого теста жюри**; ограничения перечислены в разделе 9.

1.2 Данные конкурса

Файл	Строк	Состав
<code>train.csv</code>	9 556	image_id, bbox, vehicle_id, camera_id; 1 541 идентичность, 96 камер
<code>test_query.csv</code>	1 110	image_id, bbox
<code>test_gallery.csv</code>	750	image_id, bbox
<code>images/</code>	11 416 JPEG	кадры 1920×1080, одно размеченное ТС на кадр

Источник: docs/03-dataset.md, разделы 1–2.

Главные трудности, выявленные при разборе данных (docs/03-dataset.md, раздел 4): одна и та же машина спереди и сзади выглядит совершенно по-разному; 13 % кропов касаются края кадра; около 12–14 % кропов тёмные (ночь); каршеринговые машины с одинаковой ливреей образуют трудные негативы; номера размыты прямоугольником. По ответам организаторов (docs/07-organizers-qa.md, раздел 3) в закрытом тесте около 20 % запросов не имеют пары в галерее, а совпадения в пределах одной камеры исключаются из оценки (junk-фильтр).

2 Функциональная архитектура

2.1 Пользовательские сценарии

Сценарий	Что делает пользователь	Результат
Поиск по кадру	Оператор загружает кадр и рисует/ вводит bbox	Топ-N кандидатов галереи с оценкой уверенности; решение «принять / отказ» по порогу
Отказ от идентификации	—	Если оценка топ-1 ниже порога, выдаётся отказ; ближайшие кандидаты всё равно показаны с меткой «ниже порога»
Сверка деталей	Оператор открывает пару «запрос — кандидат»	Совпавшие области (взаимно ближайшие патч-токены того же ViT) и local_score
Экспорт	Кнопка «Экспорт CSV»	CSV поиска: gallery_id, score, confidence, plate
Ведение галереи	Добавление кадра, массовый импорт по CSV, привязка ГРЗ из внешнего источника	Запись с кропом, эмбедингом и версией модели
Пакетный режим жюри	docker run ... reid-submit	Три файла сдачи для всего набора
Маршрут (симуляция)	Экран /route/<vehicle_id>	Демонстрация продуктового слоя «время-пространство»; камеры и время смоделированы, в модели и поиске не участвуют

Источники: docs/demo.md, docs/openapi.json, docs/05-scaling.md (раздел 2, пространственно-временной слой).

2.2 REST API сервиса

API описан спецификацией OpenAPI (docs/openapi.json, генерируется из кода api-gateway библиотекой utoipa; Swagger UI — /api/docs). Ошибки всегда имеют вид { error: { code, message } }.

Метод и путь	Назначение
POST /api/search	Кадр + bbox → поиск в галерее → кандидаты с карточками
GET /api/searches/{id}/compare/{gallery_id}	Сверка пары по патч-токенам, совпавшие области
GET /api/searches/{id}/export.csv	CSV результата поиска
POST /api/embed	Эмбединг одного bbox (для замера скорости по HTTP)
GET, POST /api/gallery	Список галереи; добавление ТС (кадр + bbox)
POST /api/gallery/import	Массовый импорт кадров датасета по CSV
GET /api/gallery/{id}	Одна запись галереи
PUT /api/gallery/{id}/plate	Привязать или снять ГРЗ из внешнего источника
GET /api/health, GET /api/info	Готовность сервисов; метаданные загруженной модели

Источник: docs/openapi.json.

2.3 Два пути исполнения

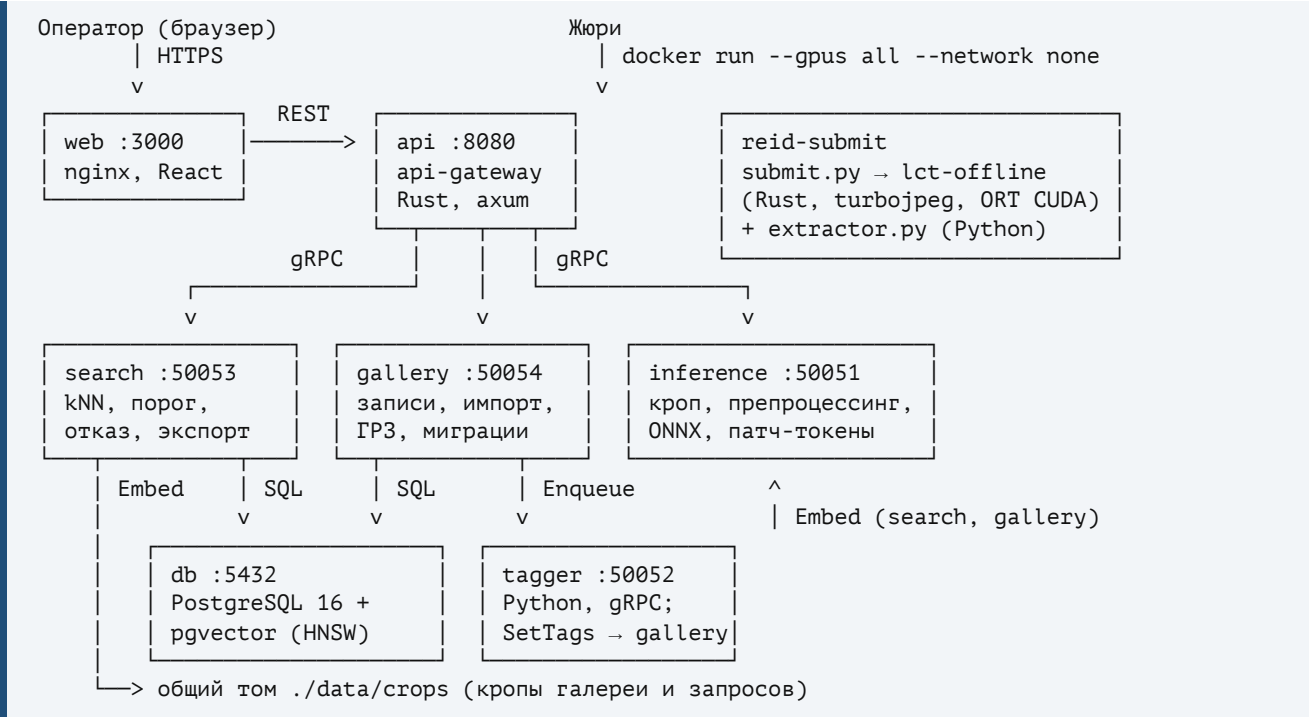
	Офлайн-образ жюри reid-submit	Сервис оператора
Вход	Каталог кадров + CSV query/gallery	Один кадр + bbox от оператора
Инференс	Нативный Rust + ONNX Runtime 1.24.4 CUDA; Python <code>extract()</code> в том же образе	Rust-сервис <code>inference</code> , ONNX Runtime (CPU по умолчанию)
Поиск	Перебор по галерее, статический DBA только по галерее ($k=4$, $\alpha=2$)	pgvector HNSW, обычный косинус
Порог отказа	0.720700740814209 (DBA)	0.6959864497184753 (косинус)
Сеть, БД	Не нужны (<code>--network none</code>)	PostgreSQL 16 + pgvector
GPU	Обязателен, без тихого перехода на CPU	Необязателен

Источники: `models/model.json`, `ml/MODEL_GUIDE.md`, `deploy/submit.Dockerfile`, `crates/inference/src/infrastructure/repositories/ort_embedding_repository.rs`.

Пороги двух путей откалиброваны отдельно и **не взаимозаменяемы**. Векторы разных версий модели в одной галерее не смешиваются: при смене модели галерея переиндексируется (`crates/gallery/migrations/0002_vitl_1024.sql`).

3 Компонентная архитектура

3.1 Сервисы и контейнеры



Сервис	Порт	Ответственность	Не делает
web	3000	SPA (React/Vite) за nginx, проксирует <code>/api</code> на шлюз	Бизнес-логики нет

Сервис	Порт	Ответственность	Не делает
api (api-gateway)	8080	REST + OpenAPI, валидация входа, маршрутизация в gallery/search, POST /api/embed напрямую в inference, отдача кропов	Не обрабатывает изображения, не ходит в БД
inference	50051	gRPC Embed, Info : кроп, препроцессинг, ONNX, патч-токены по флагу	Без состояния, не знает о галерее
search	50053	gRPC Search, Compare, Export : kNN, порог, отказ, история поисков	Не пишет в галерею
gallery	50054	gRPC Add, Import, Get, List, SetPlate, SetTags, Delete, Reindex; миграции схемы	Не ищет и не ранжирует
tagger	50052	gRPC Enqueue : теги деталей, результат в gallery.SetTags	Не ходит в БД; сейчас активен NullBackend (тегов не выдаёт)
db	5432	PostgreSQL 16 + pgvector, HNSW по косинусу	—

Источники: `docker-compose.yml`, `docs/04-architecture.md` (разделы 3–5), `tagger/backend.py`. В `docs/04-architecture.md` описано и целевое S3-хранилище кропов; в текущем `docker-compose.yml` кропы лежат на общем томе `./data/crops` и отдаются шлюзом как `/files/crops/...`

3.2 Поток запроса «поиск»

1. `api-gateway` проверяет вход (JPEG/PNG, кадр ≤ 3 МиБ, `bbox` в границах кадра) и вызывает `search.Search(image, bbox, top_n, details)`.
2. `search` вырезает кроп функцией `crop()` из крейта `inference` (тот же код, что у модели), сохраняет его и передаёт байты в `inference.Embed`.
3. kNN по HNSW среди записей текущей `model_version`: `score = 1 - (embedding <=> query)`, т. е. косинусное сходство (`crates/search/src/infrastructure/repositories/sqlx_gallery_read_repository.rs`).
4. Уверенность `confidence = \alpha \cdot score + (1 - \alpha) \cdot local\_score`; в манифесте `alpha = 1.0`, поэтому решение принимается по глобальному косинусу, а патч-токены служат для объяснения в «Сверке деталей» (`crates/search/src/domain/manifest.rs`, `models/model.json`).
5. Если `confidence` топ-1 ≥ 0.6959864497184753 — кандидат принят, иначе отказ. Результат записывается в таблицу `searches` (аудит поисков).

3.3 Контракты и хранение

- `proto/reid/v1/*.proto` — единственный источник правды для gRPC (Rust-код генерируется при сборке через `protoc`, без внешнего `protoc`; Python-стабы для `tagger` — `just proto-py`).
- `models/model.json` — манифест модели: вход/выход, размер 256×256 , mean/std, интерполяция, размерность 1 024, версия, оба порога, параметры DBA. Его читают `inference`, `search`, `api-gateway`, образ жюри и тесты паритета.
- Схема БД (владелец — `gallery`, миграции `crates/gallery/migrations/`): `gallery_items` (`bbox`, `crop_uri`, `embedding vector(1024)`, `model_version`, `plate`), `tags`, `searches`. Индекс `hnsu` (`embedding vector_cosine_ops`).

3.4 Офлайн-образ жюри

`deploy/submit.Dockerfile` собирает два этапа: Rust 1.94.1 (bookworm) компилирует нативный экстрактор `ml/inference/rust` (крейт `lct-inference`) и драйвер `ml/inference/offline` (`lct-offline`) с `--locked` и прогоняет их тесты; финальный образ на Python 3.12.12-slim содержит закреплённые CUDA-колёса (`ml/deployment/python-freeze-linux.txt`), веса `models/model.onnx`, манифест и `policy.json`. Точка входа `ml/deployment/submit.py` проверяет SHA-256 весов, пишет конфигурацию драйвера и запускает `lct-offline` через `launch-offline.py`, который подключает закреплённую `libonnxruntime.so.1.24.4`. Второй вход — `ml/extractor.py` (`extract(path, bbox)`) на том же графе и ORT CUDA. В образе заданы `LCT_REQUIRE_CUDA=1`, `CUDA_DISABLE_PTX_JIT=1`, `HF_HUB_OFFLINE=1`.

Сервисный граф `inference` отличается от графа жюри только вторым выходом `patches` (выход финального LayerNorm без CLS и register-токенов); он собирается при сборке образа (`deploy/service_model.py`), глобальный эмбединг остаётся побитно тем же.

3.5 Структура репозитория

Путь	Содержимое
<code>crates/</code>	Rust workspace: <code>api-gateway</code> , <code>inference</code> , <code>inference-client</code> , <code>search</code> , <code>gallery</code> , <code>storage</code> , <code>common</code> , <code>proto</code> , <code>cli</code> (<code>reid-cli submit</code> — проверка паритета сервиса с артефактом жюри)
<code>ml/</code>	Обучение (<code>train/</code>), нативный экстрактор (<code>inference/</code>), образ жюри (<code>deployment/</code>), <code>extractor.py</code> , валидация (<code>validation/</code>), скрипты проверки
<code>web/</code>	React + TypeScript + Vite, nginx
<code>tagger/</code>	Python gRPC-сервис тегов
<code>models/</code>	<code>model.json</code> , <code>policy.json</code> , <code>model.onnx</code> (Git LFS), лицензия DINOv3
<code>deploy/</code>	<code>rust.Dockerfile</code> , <code>submit.Dockerfile</code> , <code>service_model.py</code> , <code>vm/</code> (стенд)
<code>submission/</code>	Три файла сдачи и <code>run.json</code> с хешами
<code>bench/</code>	ANN-бенчмарк <code>pgvector</code> на 10^6 векторов

4 Методы обработки данных и алгоритмы

4.1 Предобработка

- `bbox` ограничивается границами кадра; кадр декодируется в RGB.
- Кроп растягивается (`stretch`, без сохранения пропорций) до **256×256** бикубической интерполяцией, совместимой с Pillow.
- Нормализация каналов: `mean` [0.485, 0.456, 0.406], `std` [0.229, 0.224, 0.225].

В нативном пути JPEG декодируется только по нужной области (с запасом в один iMCU для интерполяции цветности), а бикубический ресайз реализован непрерывными циклами с исходными фиксированными коэффициентами и правилами отсечения. Все **11 416** кропов побайтно совпадают с эталонным RGB-выходом; время предобработки снизилось с **9.8062 до 5.6202 мс** (−4.186 мс, 42.69 %).

Источники: `ml/MODEL_GUIDE.md`, `models/model.json`, `docs/speed.md`.

4.2 Энкодер

DINOv3 ViT-L/16 — самообучаемый vision transformer (patch 16, вход 256×256) с сильными универсальными признаками формы и текстуры, устойчивыми к изменению освещения и ракурса. Эмбеддинг — CLS-токен, пропущенный через обученный **BN neck**, затем L2-нормализация до **1 024** измерений. Классификационная голова обучения при инференсе удалена. Граф ONNX смешанной точности: веса и активации в FP16, чувствительные операции и residual-ветви в FP32. Размер графа **607 325 498 байт** (лимит ТЗ — 2 ГБ), SHA-256 `a1a63bcbfc00388ad6612a33c4861ffae8684a67bb3c0f1cb0d380b0b4bec1e0`.

Согласие FP32 PyTorch и Linux ONNX CUDA по косинусу — не ниже **0.99999717** на нулевом тензоре B1, реальных кропах FIT B1 и B17.

Источники: `ml/MODEL_GUIDE.md`, `models/README.md`, `docs/submission-readiness.md`.

4.3 Обучение: цепочка и гиперпараметры

Этап	Данные	Объём	Ключевые параметры
0. Foundation	DINOv3 ViT-L/16, timm, ревизия 30c1109	1 212 347 640 байт	Публичные веса
1. VERI-Wild TRAIN E2	277 797 изображений, 30 671 ID	2 эпохи, 34 726 обновлений	B16 = P4×K4
2. TRAIN + публичный TEST E1 (TT)	416 314 изображений, 40 671 ID	1 эпоха, 26 020 обновлений	seed 20260923; 30 671 строк классификатора и моменты AdamW переиспользованы, 10 000 новых
3. FIT-v8 (конкурс)	5 722 кропа, 928 ID	8 эпох, 2 864 обновления	seed 20260920, B16, сэмплер 2+2 по камерам, BF16, AdamW LR 3e-5 / 3e-4 (бэкбон / голова), weight decay 0.05, warmup 50 шагов, label smoothing 0.1, triplet margin 0.3, clip 5
4. Экспорт	CPU ONNX + конвертер float16 ORT 1.24.4	—	Проверка численного паритета и размещения на CUDA

Источники: `ml/TRAINING.md`, `ml/MODEL_GUIDE.md`.

Функция потерь на этапе FIT — классификация с label smoothing плюс batch-hard triplet loss. Батч 16 = 4 идентичности × 4 кадра; кадры идентичности берутся в первую очередь с разных камер, а для идентичностей, снятых двумя камерами (не меньше двух кадров на каждой), блок составляет как 2 + 2 кадра с двух камер (`ml/train/reid_model.py`, `FitTwoPlusTwoBatches`). Так модель учится сближать разные ракурсы и условия съёмки одной машины. `camera_id` используется **только** сэмплером и официальным оценщиком, в энкодер не подаётся. Историческая длительность этапа TT — 115.306 минуты; повторный FIT-этап 29.09 занял 809.124 с обучения при пике 76 °C.

4.4 Роли данных и проверка чистоты

Роль	Назначение	Состав
FIT	Обучение	5 722 кропа / 928 ID (FIT-v8)

Роль	Назначение	Состав
SEARCH	Выбор модели (многократно)	215 запросов (172 с парой + 43 без пары), 750 в галерее
Calibration	Только выбор порога после фиксации модели	200 запросов (160 + 40), 750 в галерее
Audit	Отчёт о замороженной модели, без настройки	200 запросов (160 + 40), 750 в галерее
Jury test	Только финальный инференс	1 110 query + 750 gallery, меток нет

Источники: `ml/validation/{search,calibration,audit}/*.csv`, `docs/submission-readiness.md`, `CONTEXT.md`.

Роли не пересекаются ни по изображениям/пикселям, ни (для размеченных ролей) по идентичностям и сценам. Все **11 416** конкурсных и **416 314** внешних изображений перехешированы: расхождений хешей источников — 0, точных пересечений по закодированным файлам, полному RGB и кропам — 0. Точное сравнение не исключает перцептивных дублей и пересечений с корпусом предобучения foundation-модели. Исторически FIT-v8 сохранил 5 722 из 5 746 строк FIT (24 проверенных кропа исключены до введения правила «без чистки»); новых исключений и переразметки нет.

4.5 Ранжирование: DBA только по галерее

В офлайн-пути галерея один раз уточняется статическим database-side augmentation (DBA). Для каждого вектора галереи g_i берутся он сам и $k = 4$ ближайших соседа $N_k(i)$ в галерее:

$$\tilde{g}_i = \frac{\sum_{j \in \{i\} \cup N_k(i)} \max(0, \cos(g_i, g_j))^\alpha g_j}{\left\| \sum_{j \in \{i\} \cup N_k(i)} \max(0, \cos(g_i, g_j))^\alpha g_j \right\|}, \quad \alpha = 2.$$

Каждый запрос q независимо сравнивается с преобразованной галереей: $s(q, i) = \cos(q, \tilde{g}_i)$; равные значения упорядочиваются по порядку `test_gallery.csv`. Расширения запросов, кластеризации запросов, состояния между запросами, OCR и метаданных нет — это соответствует потоковому протоколу организаторов (`docs/07-organizers-qa.md`, раздел 2). В `embeddings.npy` пишутся исходные векторы, а не DBA. Проверка независимости: обращение порядка запросов и прогон по одному сохранили все ранжирования и решения; 31/33 векторов одинаковой формы батча и все векторы галереи совпали побайтно, при смене формы батча косинус ≥ 0.999998388 (разница оценок ≤ 0.000015498).

Источники: `ml/MODEL_GUIDE.md`, `docs/submission-readiness.md`, `ml/validation/final_20260929/query-independence.json`.

4.6 Режим отказа

В `submission.csv` всегда пишутся 10 кандидатов, даже при отказе. В `candidates.csv` строка топ-1 появляется только если его оценка $\geq$ порога; отказ — отсутствие строк для запроса. Оценки — не вероятности. Обоснование порога — в разделе 7.

4.7 Объяснимость

«Сверка деталей» строит пары взаимно ближайших патч-токенов того же ViT для запроса и кандидата (`crates/search/src/domain/patch_match.rs`) и показывает совпавшие области и `local_score`. Это объяснение без отдельной модели; на решение о принятии при `alpha = 1.0` не влияет.

5 Сборка, установка и запуск

5.1 Требования

- Docker с Docker Compose v2; Git LFS; just (для команд образа жюри).
- Для образа жюри: Linux x86_64, GPU NVIDIA с CUDA, NVIDIA Container Toolkit (`--gpus all`). Интернет нужен только при `docker build`; запуск — с `--network none`.
- Кадры датасета кладутся в `dataset/images/` (в git не входят, около 7.5 ГБ).

5.2 Сервис оператора одной командой

```
git clone <репозиторий> lct-2026 && cd lct-2026
git lfs install && git lfs pull # models/model.onnx, 607 325 498 байт
cp .env.example .env && docker compose up --build
# интерфейс: http://localhost:3000, Swagger: http://localhost:3000/api/docs
# импорт галереи из смонтированного датасета:
curl -F csv=@dataset/test_gallery.csv -F images_dir=images
http://localhost:3000/api/gallery/import
```

Сборка образа `inference` проверяет SHA-256 весов против манифеста (`deploy/rust.Dockerfile`). Необязательное GPU-переопределение: `docker compose -f docker-compose.yml -f docker-compose.gpu.yml up --build`.

5.3 Офлайн-образ жюри

```
git lfs install
git lfs pull
just submit-image # docker build --platform linux/amd64 -f deploy/submit.Dockerfile -t reid-submit .
docker run --rm --network none --gpus all \
  -v /absolute/path/to/data:/data:ro -v "$PWD/submission:/out" reid-submit \
  --images /data/images --query /data/test_query.csv --gallery /data/test_gallery.csv --out /out
python ml/scripts/verify_submission.py --submission submission \
  --query dataset/test_query.csv --gallery dataset/test_gallery.csv --manifest models/model.json
just perf-gpu # замер скорости по протоколу жюри + двукратная побайтовая проверка
```

`verify_submission.py` независимо проверяет формат, конечность и нормировку float32-векторов, все топ-10, точное DBA-ранжирование и замороженные решения отказа. Та же проверка встроена в образ (`/opt/lct/verify_submission.py`). `just perf-gpu` требует несуществующий каталог вывода (по умолчанию `perf-gpu/`).

5.4 Воспроизведение обучения и калибровки

```
just train-check # проверка входов без GPU
just train-veriwild /data/VERI-Wild /data/dinov3.safetensors
cd ml && python -m train.stage1_tt ... # полный вызов в ml/TRAINING.md
python -m train.stage2_fit --config train/configs/stage2_fit_v8_2plus2.json \
  --weights runs/stage1_tt/backbone.safetensors --run-dir runs/stage2_fit
cd .. && just train-export runs/stage2_fit/checkpoints/epoch0008-batch0000-step002864.pt
cd ml && python scripts/calibrate_native.py --submission /data/calibration-output \
  --episode validation/calibration --json /data/calibration-replay.json
```

Окружение обучения: Python 3.12, torch 2.14.0+cu126, torchvision 0.29.0+cu126, timm 1.0.29, safetensors 0.8.0, GPU Ampere с BF16 (исходно RTX 3090). Побитно одинаковое переобучение не обещается: первый шаг совпал по потере и градиенту, дальше CUDA-обучение численно

расходится.

5.5 Тесты и линтеры

```
cd ml && uv sync --group dev && uv pip install -r requirements-submit.txt && cd ..
just test && just lint
cd web && npm test && npm run build
```

Итоговые проверки: 48 Python-тестов; 148 тестов Rust workspace + 1 тест паритета на реальной модели; 28 нативных модульных и 13 офлайн-тестов; 81 веб-тест со сборкой; 8 тестов tagger (docs/AGENT_CONTEXT.md, docs/submission-readiness.md).

6 Метрики на валидации

6.1 Протокол

Оценка повторяет протокол жюри (docs/07-organizers-qa.md, раздел 2): mAP@10 по первым 10 кандидатам, AP нормируется на $\min(n_{pos}, 10)$, усреднение по запросам; объекты галереи с тем же vehicle_id и той же камерой удаляются до усечения до 10; запросы без валидной пары исключаются из mAP/Rank и оцениваются только в режиме отказа. F1 и TNR — на уровне запроса по верхнему кандидату. Неизменённый оценщик организаторов — ml/organizers/evaluate.py (SHA-256 655c71db8c2e4d2cd7680c40c768afacfdffff360401111c1a46df921551ffa3).

6.2 Результаты выбранной модели

Роль	Ранжирование	mAP@10	Rank-1	Rank-5	TP / FP / FN / TN	F1	TNR	PR-AUC	Q
SEARCH	DBA	92.0048	91.28	95.93	148 / 0 / 24 / 43	0.9250	1.0000	0.9839	0.9475
Calibration	DBA	82.9999	79.38	91.88	142 / 0 / 18 / 40	0.9404	1.0000	0.9851	0.9583
Audit	DBA	87.0033	87.50	93.13	141 / 2 / 19 / 38	0.9307	0.9500	0.9839	0.9365
SEARCH	косинус	90.4849	91.86	97.09	145 / 1 / 27 / 42	0.9119	0.9767	0.9806	0.9314
Calibration	косинус	80.3487	78.75	93.75	142 / 0 / 18 / 40	0.9404	1.0000	0.9842	0.9583
Audit	косинус	85.2397	85.63	93.13	140 / 1 / 20 / 39	0.9302	0.9750	0.9828	0.9437

Источник: ml/validation/holdout.json (Linux native CUDA, финальный граф; значения округлены). mAP@10, Rank-1, Rank-5 — в процентах.

Условный bootstrap по идентичностям на audit (5 000 повторов, 158 известных идентичностей, фиксированная масса 160 : 40): 95 %-интервалы **82.316–91.139 % mAP@10** и **0.9003–0.9656 Q**. Интервал не поправляет на адаптивный выбор модели и не предсказывает тест жюри.

6.3 История выбора модели

Свидетельство	mAP@10	Q	Комментарий
Исходный TT, seed 20260920, BF16	91.956359 %	0.958882	Исторический SEARCH, development-порог
Исходный TT, seed 20260921, BF16	90.406911 %	0.961047	Второй seed на SEARCH
Повторный FIT, seed 20260920	91.212704 %	0.965854	8 эпох, 2 864 обновления
Выбранная модель, Linux native SEARCH	92.004809 %	0.947500	Финальный откалиброванный порог
Выбранная модель, audit	87.003299 %	0.936485	Без последующей настройки
Предыдущая модель, audit	84.522693 %	0.958077	Для сравнения

Источники: docs/submission-readiness.md, docs/adr/0004-frozen-tt-submission.md.

Два seed-а улучшили SEARCH относительно предыдущей модели на 0.864 и 0.279 п. п.; средний парный выигрыш 0.5715 п. п. с 95 %-интервалом $[-0.62, 1.95]$, который включает ноль. Выбор — решение разработки, а не доказательство статистического превосходства. На audit mAP вырос на 2.4806 п. п., а Q снизился с 0.958077 до 0.936485; по линейному development-прокси $45 \cdot \text{mAP} + 10 \cdot Q$ это около +0.90 балла, но реальное преобразование в баллы жюри не установлено.

7 Обоснование порога отказа

7.1 Постановка

Пусть для запроса q модель выдаёт верхнего кандидата $c(q)$ с оценкой $s(q)$ (DBA-косинус в офлайн-пути). Решение при пороге t : принять $c(q)$, если $s(q) \geq t$, иначе отказ. Счётчики по определениям организаторов (docs/07-organizers-qa.md, раздел 3):

- TP — пара есть, ответ дан и верхний кандидат верен;
- FP — ответ дан, но верхний кандидат неверен или пары нет;
- FN — пара есть, ответа нет;
- TN — пары нет, отказ.

$$F1(t) = \frac{2 TP}{2 TP + FP + FN}, \quad TNR(t) = \frac{TN}{TN + FP_{\text{без пары}}}, \quad Q(t) = 0.7 F1(t) + 0.3 TNR(t).$$

Баллы жюри за режим отказа равны $10\% \times (0.7 \cdot F1 + 0.3 \cdot TNR)$, поэтому максимизация Q напрямую максимизирует этот критерий.

7.2 Почему именно так выбирается порог

1. **Совпадение априорной доли.** В закрытом тесте около 20 % запросов без пары; эпизод калибровки собран с той же долей: 160 запросов с парой и 40 без пары.

- 2. Точный argmax.** $Q(t)$ кусочно-постоянна и меняется только в наблюдаемых значениях $s(q)$. Поэтому достаточно перебрать все точные наблюдаемые границы и два крайних решения («принять всё», «отказать всем») и взять максимум. Ничьи разрешаются в пользу большего TNR, затем большего порога (консервативнее к ложным ответам). В отличие от исторического CPU-помощника, берётся точная наблюдаемая граница, а не середина промежутка.
- 3. Порядок заморозки.** Модель зафиксирована до калибровки, пороги — до audit; audit не менял ни модель, ни предобработку, ни порог. `ml/scripts/calibrate_native.py` воспроизводит замороженные значения из векторов калибровки.

7.3 Результат

Путь	Порог	Calibration TP/FP/FN/TN	F1	TNR	Q	Audit TP/FP/FN/TN	Audit Q
Офлайн, DBA k=4, alpha=2	0.720700740814209	142 / 0 / 18 / 40	0.9404	1.0	0.9583	141 / 2 / 19 / 38	0.9365
Сервис, обычный косинус	0.6959864497184753	142 / 0 / 18 / 40	0.9404	1.0	0.9583	140 / 1 / 20 / 39	0.9437

Источники: `models/model.json`, `ml/validation/holdout.json`, `docs/submission-readiness.md`, `ml/validation/final_20260929/threshold-freeze.json`.

Проверка арифметики для DBA на калибровке: $F1 = 284 / (284 + 0 + 18) = 0.9404$, $TNR = 40 / 40 = 1$, $Q = 0.7 \cdot 0.9404 + 0.3 \cdot 1 = 0.9583$. На audit: $F1 = 282 / 303 = 0.9307$, $TNR = 38 / 40 = 0.95$, $Q = 0.6515 + 0.285 = 0.9365$. Порог выбран в точке, где на калибровке ещё нет ни одного ложного принятия, а потеря полноты (18 отказов на запросах с парой) обходится дешевле, чем ложные ответы: FP штрафуются и в F1, и в TNR. На audit порог дал 2 ложных принятия из 40 запросов без пары — это ожидаемый перенос на новые данные, а не повод перенастраивать порог.

Оценки не являются вероятностями; в `candidates.csv` значение `confidence` монотонно и сравнивается с порогом. Порог привязан к версии модели 1.2.0-final: для новых весов нужна новая калибровка на подходящих данных.

8 Скорость

8.1 Протокол

По ответам организаторов (`docs/07-organizers-qa.md`, раздел 1): таймер включает чтение файла, декодирование, кроп, препроцессинг, forward и L2-нормализацию, без поиска; latency — медиана 300 вызовов batch=1 после 50 прогревочных; throughput — лучший устойчивый FPS при батчах 1/8/16/32, не короче 10 с на размер. Полный балл: latency ≤ 40 мс и throughput ≥ 100 FPS. Стенд жюри: RTX A5000, драйвер CUDA 12.2. `just perf-gpu` воспроизводит этот протокол в образе.

8.2 Результаты (RTX 3090, драйвер 616.92, Docker Linux, сеть отключена, PTX JIT отключён)

Путь	V1 медиана	V1 p95	Лучший устойчивый FPS
Нативный (первый прогон)	22.417 мс	30.337 мс	181.549

Путь	B1 медиана	B1 p95	Лучший устойчивый FPS
Нативный (повтор из каталога сдачи)	24.575092 мс	29.637462 мс	185.935319
Python <code>extract()</code> (первый прогон)	24.99 мс	33.305 мс	119.0
Python <code>extract()</code> (повтор из каталога сдачи)	27.65 мс	—	110.5

Источники: `docs/speed.md`, `docs/submission-readiness.md`,
`ml/validation/final_20260929/{native,python,submission-folder-*}.json`.

Пик температуры GPU — 64 °C, пик памяти GPU — 4 620 МиБ (повтор — 4 702 МиБ). Оба полных нативных прогона дали побайтно одинаковые три файла; два Python-прогона B1 — одинаковые векторы 300×1 024. Суммарный размер весов, сканируемых жюри, — 607 325 739 байт (граф плюс 241-байтный тестовый ONNX-фикстур) при лимите 2 ГБ. Локальные результаты не являются измерением на A5000.

9 Анализ ошибок и ограничения

9.1 Характерные ошибки

- **Перед против зада.** SEARCH, идентичность 1258: после удаления junk-пар первый верный (фронтальный) вид стоит на 9-м месте; в топ-10 попал один из двух валидных позитивов (`ml/validation/final_20260929/error-example.json`). Один автомобиль спереди и сзади почти не имеет общих видимых деталей.
- **Почти одинаковые машины.** Похожий вид сзади другого ТС (пример с G-классом в презентации) может получить высокое сходство: высокая оценка не гарантирует идентичность. Сюда же относятся каршеринговые машины одной ливреи (`docs/03-dataset.md`).
- **Перекрытие и освещение.** Разбор development-данных нашёл смешанные bbox, частично видимые машины и расхождения «ночь/день»; метки при этом не менялись.
- **Ошибки режима отказа.** На audit 19 запросов с парой получили отказ (FN), 2 запроса без пары — ложное принятие (FP); на SEARCH — 24 FN и 0 FP (`ml/validation/holdout.json`).
- **Контроль зоны номера.** Приблизительное закрашивание зон номеров на SEARCH снизило mAP с 92.0048 % до 91.2924 % (−0.7124 п. п.); случайные маски той же площади — 91.8018 %. Зоны найдены на 470 из 965 кропов, в среднем 4.20 % площади кропа. Это не точный тест жюри с масками и не доказательство нулевой зависимости от зоны номера (`docs/submission-readiness.md`).

9.2 Пути развития

Калибровка порога отдельно по условиям (ночь, дождь, ракурс); переранжирование по деталям (патч-токены уже доступны); пространственно-временная проверка «успел ли доехать» при наличии камеры и времени; обратная связь операторов как трудные пары для дообучения (`docs/05-scaling.md`, раздел 2).

9.3 Ограничения, которые мы не скрываем

- SEARCH многократно использовался для выбора модели. Calibration и audit открывались 27 сентября для предыдущей модели, поэтому это **не нетронутые отложенные выборки**.
- Метрики и место на скрытом тесте жюри неизвестны; линейный балл — только development-прокси.
- Машина с RTX A5000 и драйвером 535 была недоступна. Образ содержит CUDA 12.9 runtime; по документации NVIDIA минорная совместимость CUDA 12.x начинается с драйвера 525.60.13, но отключение PTX на новом драйвере не эмулирует 535. Совместимость не гарантируется.
- Исторические 24 исключения из FIT-v8 раскрыты; повторное обучение воспроизвело только FIT-этап от проверенного TT-родителя (91.212704 %, -0.743655 п. п. к исходному максимуму), внешние этапы не повторялись.
- Точные проверки пересечений не исключают перцептивных дублей и пересечения с корпусом предобучения DINOv3.
- Сервисный `tagger` сейчас работает с `NullBackend` и тегов не выдаёт; экран маршрута — симуляция на смоделированных камерах.

10 Масштабирование

10.1 ANN на 10^6 векторов

Замер `just ann-bench : pgvector 0.8.6, HNSW, 106 синтетических векторов 512-d` (166 666 «машин» × 6 кадров), Docker на Apple M4 Pro.

Конфигурация	Индекс	Перебор p50	HNSW p50 / p95	recall@10
m=16, ef_construction=64, shared_buffers 128 МБ, ef_search=200	731 с, 2.5 ГБ	1 273 мс	21 / 36 мс	0.50
m=24, ef_construction=128, shared_buffers 2 ГБ, ef_search=100	1 669 с, 2.5 ГБ	1 733 мс	33 / 60 мс	0.79
то же, ef_search=400			44 / 89 мс	0.85

Источники: docs/05-scaling.md, раздел 5; bench/results/.

HNSW даёт порядка 40-кратного выигрыша против перебора на одном узле, но при миллионе векторов нужны $m \geq 24$ и $ef_construction \geq 128$, а индекс и таблица должны помещаться в RAM. Замер сделан на 512-d синтетике; для выбранной модели с 1 024 измерениями его нужно повторить.

10.2 Эталонная архитектура для города

- **Шина событий** (Kafka / NATS JetStream) между камерными комплексами и воркерами эмбеддингов.
- **GPU-воркеры с dynamic batching** (например, Triton) за тем же контрактом `inference`.
- **Векторный индекс** (Qdrant / Milvus) с партициями по времени и региону и квантизацией; PostgreSQL — для событий, идентичностей, ГРЗ и аудита.
- **Объектное хранилище кропов** с политикой хранения.

- **Порог отказа по условиям**, пространственно-временной фильтр двойников, жизненный цикл модели (реестр, теневой прогон, переиндексация при смене версии), RBAC и аудит поисков.

Границы сервисов и контракты (`proto/`, `models/model.json`, REST) совпадают с эталоном; в демо заменены три компонента (Triton → `ort` в процессе, Qdrant/Milvus → `pgvector`, шина → синхронный вызов), и замена каждого не меняет ни `proto`, ни API (`docs/05-scaling.md`, раздел 3).

11 Внешние библиотеки, фреймворки и датасеты

Датасеты и предобученные веса

Источник	Версия / объём	Лицензия / условия	Использование
DINOv3 ViT-L/16, <code>timm vit_large_patch16_dinov3.lvd1689m</code>	ревизия <code>30c1109559f65dea34316b0d4842d35c5771fe11</code> , 1 212 347 640 байт, SHA-256 <code>45172f20...7941</code>	DINOv3 License (<code>models/DINOv3_LICENSE.md</code>)	Инициализация энкодера
VERI-Wild TRAIN	277 797 изображений, 30 671 ID	Данные авторов (PKU-IMRE), некоммерческие исследования и обучение	Этапы E2 и TT
VERI-Wild публичный TEST	138 517 изображений, 10 000 ID	То же; разрешён ответом организаторов Q46	Этап TT
Датасет ЛЦТ 2026	11 416 кадров, <code>train.csv</code> 9 556 строк	Условия хакатона	FIT-v8, SEARCH, calibration, audit

Источники: `ml/TRAINING.md`, `models/README.md`, `docs/03-dataset.md`.

Образ жюри: Python-рантайм (`ml/deployment/python-freeze-linux.txt`)

Пакет	Версия	Пакет	Версия
<code>onnxruntime-gpu</code>	1.24.4	<code>nvidia-cudnn-cu12</code>	9.26.0.51
<code>numpy</code>	2.5.2	<code>nvidia-cublas-cu12</code>	12.9.2.10
<code>pillow (Python extract())</code>	12.3.0	<code>nvidia-cuda-runtime-cu12</code>	12.9.79
<code>protobuf</code>	7.36.2	<code>nvidia-cuda-nvrtc-cu12</code>	12.9.86
<code>flatbuffers</code>	25.12.19	<code>nvidia-cufft-cu12</code>	11.4.1.4
<code>sympy</code>	1.14.0	<code>nvidia-curand-cu12</code>	10.3.10.19
<code>mpmath</code>	1.3.0	<code>nvidia-nvjitlink-cu12</code>	12.9.86
<code>packaging</code>	26.3		

Образ жюри: нативный Rust (`ml/inference/rust/Cargo.toml`, `Cargo.lock`)

Крейт	Версия	Назначение
<code>ort / ort-sys</code>	2.0.0-rc.12	Привязки ONNX Runtime (load-dynamic, CUDA)
<code>turbojpeg</code>	1.5.1	Декодирование JPEG (libjpeg-turbo, SIMD)
<code>image</code>	0.25.10	PNG

Крейт	Версия	Назначение
libloading	0.9.0	Динамическая загрузка ORT
serde, serde_json, csv, sha2, anyhow, thiserror	1 / 1 / 1 / 0.10 / 1 / 2	Конфигурация, CSV, хеши

Сервисы Rust (`Cargo.toml` , версии из `Cargo.lock`)

Крейт	Версия	Крейт	Версия
tokio	1.53.1	sqlx (PostgreSQL)	0.8.6
tonic (gRPC)	0.14.6	pgvector	0.4.2
prost	0.14.4	ort (сервис, CPU)	2.0.0-rc.13
protox	0.9.1	ndarray	0.17.2
axum	0.8.9	image	0.25.10
utoipa / utoipa-swagger-ui	5.5.0 / 9.0.2	object_store	0.11.2
tower-http	0.7.1	config	0.15.25
clap	4.6.7	chrono	0.4.45
serde	1.0.229	tracing	0.1.44

Обучение и валидация (`ml/TRAINING.md` , `ml/uv.lock`)

Пакет	Версия	Пакет	Версия
Python	3.12	numpy	2.5.3
torch	2.14.0+cu126	pandas	3.0.5
torchvision	0.29.0+cu126	pillow	12.3.0
timm	1.0.29	scikit-learn	1.9.1
safetensors	0.8.0	pyyaml	6.0.3
onnx	1.23.0	tqdm	4.70.1
onnxruntime (конвертер float16)	1.24.4	pytest / ruff	9.1.1 / 0.16.7

Веб-интерфейс (`web/package.json` , версии из `web/package-lock.json`)

Пакет	Версия	Пакет	Версия
react / react-dom	19.3.0	vite	6.4.3
lucide-react	1.47.0	@vitejs/plugin-react	4.7.0
@fontsource-variable (manrope, unbounded, jetbrains-mono)	5.3.0	typescript	5.9.3
		vitest	3.2.7

Базовые образы и инфраструктура

Компонент	Версия	Где
rust (сборка образа жури)	1.94.1-bookworm, закреплён по digest	<code>deploy/submit.Dockerfile</code>

Компонент	Версия	Где
python (образ жури)	3.12.12-slim-bookworm, закреплён по digest	deploy/submit.Dockerfile
rust (сервисы) / debian	1.92-trixie / trixie-slim	deploy/rust.Dockerfile
python (сборка сервисного графа)	3.12-slim по digest; onnx 1.23.0, numpy 2.5.3	deploy/rust.Dockerfile
node / nginx	22-alpine / 1.27-alpine	web/Dockerfile
PostgreSQL + pgvector	pgvector/pgvector:pg16 (в ANN-замене pgvector 0.8.6)	docker-compose.yml
tagger	python:3.12-slim; grpcio ≥ 1.66, grpcio-health-checking ≥ 1.66, protobuf ≥ 5, pyyaml ≥ 6 (без lock-файла)	tagger/Dockerfile, tagger/pyproject.toml
Caddy (TLS на демо-стенде)	2.10-alpine	deploy/vm/docker-compose.vm.yml

12 Команда и ссылки

Участник	Роль
Дворядкин Кирилл	Капитан, тимлид: инфраструктура, API-шлюз, веб, упаковка
Глухов Сергей Владимирович	ML-инженер: модель, данные, обучение, валидация
Вологин Иван Александрович	Бэкенд-разработчик: нативный инференс, поиск, галерея, теггер

- Онлайн-демо: <https://lct.znataalk.ai>
- Презентация: docs/presentation/reid-lct2026-ru.pdf, docs/presentation/reid-lct2026-ru.pptx
- Полная проверка: docs/submission-readiness.md; числовые доказательства: ml/validation/final_20260929/; хеши файлов сдачи: submission/run.json.

Ключевые хеши: граф a1a63bcbfc00388ad6612a33c4861ffae8684a67bb3c0f1cb0d380b0b4bec1e0; манифест 1015c336d068563fad87d7a1d368a938094a2067652cf25e80b29ce80dfd1695; политика Linux f43c01750b27396dcba069413a3ef50f067875e28927f42713d71cc3e44e6adc; образ sha256:099f42bf4f21998be21592cbee536c8b28a6488dc2fc7a47566c381042a1ec6a.